Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

LOCAL EVENT ORGANIZER APP

.NET Final Project Proposal

Submitted for .NET Development Course
Academic Year 2025 – 2026

1. Project Name

Project Name: Local Event Organizer App

Platform: ASP.NET Core MVC Web Application

Technology Stack: ASP.NET Core | SQL Server | Entity Framework Core | Bootstrap

2. Problem Statement

Communities today face a significant challenge when it comes to organizing, discovering, and participating in local events. Event information is scattered across social media, physical flyers, and word-of-mouth, making it extremely difficult for residents to stay informed.

As a result, many valuable community events go unnoticed due to the lack of a centralized and accessible platform. This fragmentation leads to:

- Low attendance at community events due to poor visibility
- Difficulty for organizers to manage registrations and communicate updates
- No centralized record of events, making planning and coordination harder
- Missed opportunities for community engagement and networking

3. Objective

The primary objective of the Local Event Organizer App is to develop a web-based platform using ASP.NET Core that serves as a centralized hub for creating, discovering, and managing local events within a community.

Specifically, the application aims to:

- Provide a user-friendly interface where community members can browse and register for upcoming local events
- Allow event organizers to easily create, publish, and manage their events including attendee tracking
- Enable an admin panel for full platform oversight, including event approval and user management
- Streamline event registration by replacing manual or paper-based sign-ups with a digital process
- Improve community engagement by making local events more visible and accessible to all residents

4. Functional and Non-Functional Requirements

4.1 Functional Requirements

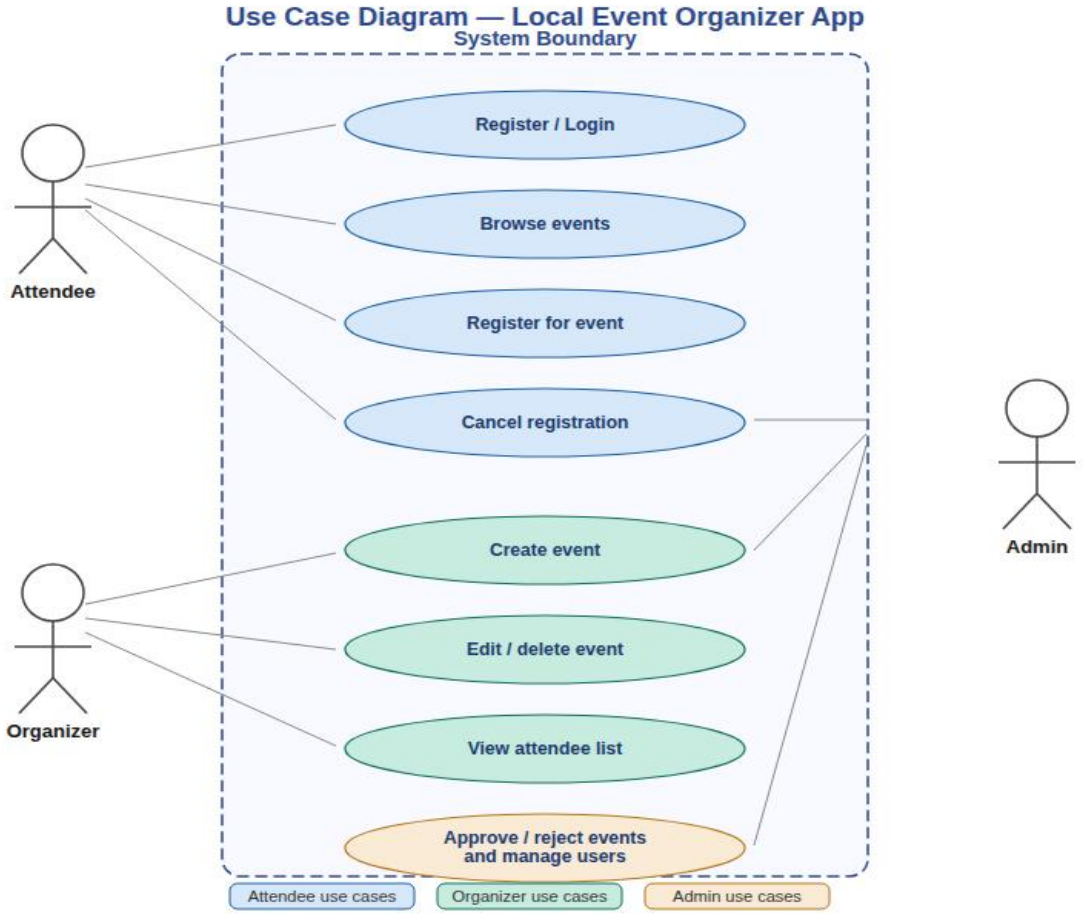
ID	Feature	Description
FR-01	User Registration & Login	Users can sign up, log in, and manage profile. Roles: Admin, Organizer, Attendee.
FR-02	Browse Events	Users can view all upcoming events with title, date, location, and category.
FR-03	Event Registration	Authenticated users can register for events and cancel their registration.
FR-04	Create & Manage Events	Organizers can create new events, edit details, and delete their own events.
FR-05	Admin Dashboard	Admin can approve/reject events, manage users, and monitor the platform.
FR-06	Event Categories	Events organized by categories (Sports, Music, Tech, Health) for easy filtering.
FR-07	Attendee List	Organizers can view the list of registered attendees for each of their events.
FR-08	Search & Filter	Users can search events by keyword, category, or date range.

4.2 Non-Functional Requirements

Category	Requirement
Performance	Pages should load within 3 seconds under normal usage conditions.
Security	Passwords must be hashed. Role-based authorization enforced across all pages.
Usability	UI must be clean, responsive, and accessible on desktop and mobile devices.
Scalability	Application should support future features and a growing number of users.
Reliability	System should handle errors gracefully with meaningful messages to users.
Maintainability	Code should follow MVC architecture and be well-organized for easy maintenance.

5. Use Case Diagram

The diagram below describes the interactions between the three main actors (Admin, Organizer, and Attendee) and the core system functionalities.



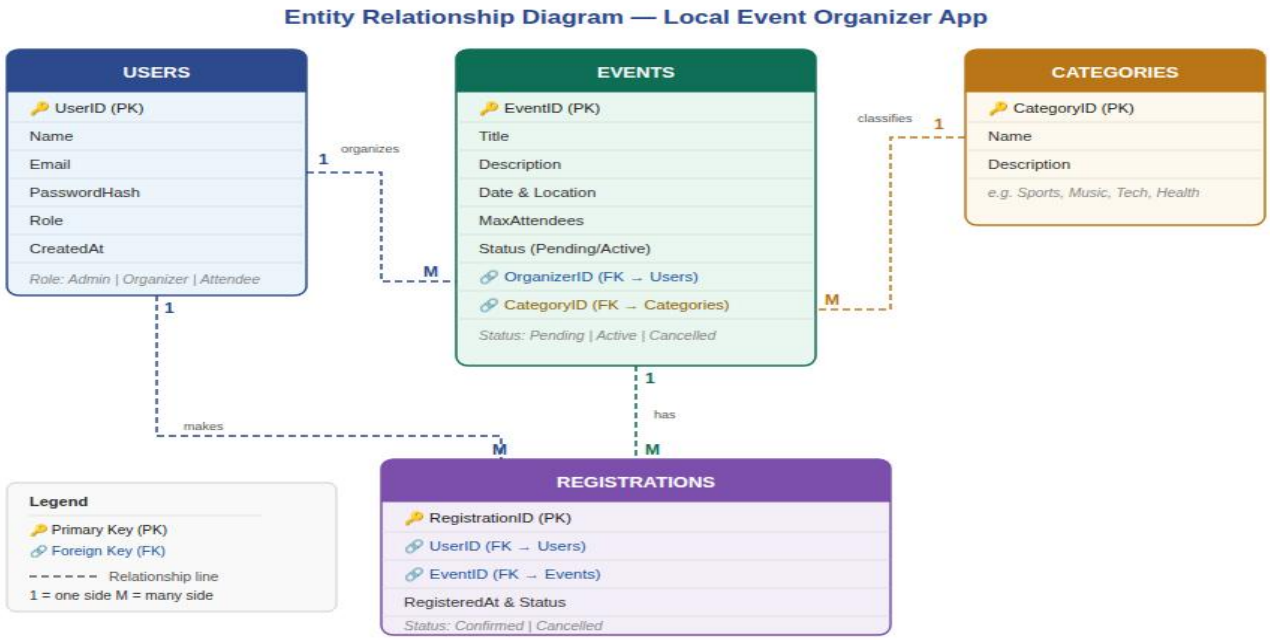
Use Case Summary Table

Actor	Use Case	Description
Attendee	Register / Login	Create account and authenticate
	Browse Events	View all available events
	Register for Event	Sign up to attend an event
	Cancel Registration	Remove own event registration
Organizer	Create Event	Add a new event to the platform
	Edit / Delete Event	Modify or remove own events
	View Attendee List	See who registered for their event
Admin	Approve / Reject Events	Control event visibility on platform

	Manage Users	View, activate, or deactivate users
	Manage Categories	Add or edit event categories

6. Database Diagram

The application uses SQL Server with 4 tables. The ERD below shows all tables, their fields, and the relationships (foreign key connections) between them.



Relationship Summary

From Table	Type	To Table	Description
Users	One-to-Many (1:M)	Events	One organizer creates many events
Users	One-to-Many (1:M)	Registrations	One user registers for many events
Events	One-to-Many (1:M)	Registrations	One event has many registrations

Categories	One-to-Many (1:M)	Events	One category classifies many events

7. Project Timeline

The project is planned over 5 weeks. Each week has a clear focus to keep development organized and on track:

Week	Phase	Tasks	Deliverable
Week 1	Planning & Setup	Finalize requirements, create ASP.NET Core project, configure SQL Server & EF Core, set up Identity and roles	Project skeleton + Auth
Week 2	Event Management	Build event CRUD for organizers: create, edit, delete events. Add categories. Organizer dashboard.	Organizer features
Week 3	User Features	Browse events, search & filter, register for event, cancel registration, view my registered events.	Attendee features
Week 4	Admin Panel	Build admin dashboard: approve/reject events, manage users and categories, view platform statistics.	Admin panel done
Week 5	Testing & Polish	Test all features end-to-end, fix bugs, improve UI with Bootstrap, finalize pages, prepare demo.	Final submission

Weekly Effort Estimate

Week	Focus	Estimated Hours
Week 1	Planning + Setup + Auth	8 – 10 hours
Week 2	Event CRUD + Categories	8 – 10 hours
Week 3	Attendee Features	7 – 9 hours
Week 4	Admin Panel	7 – 9 hours
Week 5	Testing + UI Polish	5 – 7 hours

End of Project Proposal